

Full Stack AI Engineer Assessment

Overview & Purpose

This assessment evaluates the candidate's ability to design, build, and reason about AI-powered full-stack systems in a production-oriented context.

The focus is on:

- End-to-end application design
- Integration of LLMs / AI services
- Backend + frontend coherence
- Data handling, security, and scalability
- Practical engineering judgment

Instructions

- You will complete practical and design-oriented exercises.
- Deliverables must be submitted via GitHub / GitLab.
- Time expectation: ~6–10 hours total.
- Bonus sections are optional and meant to surface seniority.
- Clarity, reasoning, and trade-offs matter more than completeness.

Part 1 — AI-Powered Full-Stack Application

1.1 Problem Statement

Build a small AI-assisted application that allows users to:

- Submit content (text or documents)
- Interact with an AI assistant over that content
- View structured outputs generated by the AI

Example use cases (pick one):

- AI assistant that answers questions about uploaded documents
- AI assistant that summarizes and classifies user input
- AI assistant that extracts structured data from free text

1.2 Backend (AI-First)

Use Java (Spring Boot) or Node.js (preferred).

Required:

- REST API
- One AI interaction endpoint (LLM call)

- One persistence layer (PostgreSQL, MongoDB, or DynamoDB)
- Authentication (JWT or similar)

AI-specific requirements:

- Clear separation between:
 - Prompt construction
 - Model invocation
 - Response post-processing
- Ability to switch LLM providers (mocked or abstracted is fine)
- Basic prompt versioning or configuration

Explain:

- How you prevent prompt injection or unsafe input
- How you'd control costs and rate limits in production

1.3 Frontend (AI-Aware UX)

Use React

Required:

- At least 2 pages (e.g. Input / Results)
- A form to submit data to the backend
- Display AI responses in a user-friendly way
- Loading, error, and empty states

AI-specific UX considerations:

- Show model status (thinking, partial results, errors)
- Allow user to refine or re-ask a question
- Handle hallucinations or uncertainty gracefully

Design quality is secondary to usability and clarity.

Part 2 — AI Data & Architecture Thinking

2.1 Data Flow & Storage

Explain (code or README):

- What data you store vs what you don't
- How long AI inputs/outputs are retained
- How you would handle: PII, Logging, Auditability

Bonus:

- Simple vector store integration (FAISS, Pinecone, OpenSearch, etc.)
- Embeddings + retrieval-augmented flow (RAG), even if minimal

2.2 AI Evaluation & Reliability

You don't need to build a full evaluation system, but explain:

- How you would measure output quality
- How you'd detect regressions after prompt/model changes

- How you'd handle "AI gives wrong answer" in production

Part 3 — Infrastructure & Deployment

3.1 Cloud & Runtime

Use AWS

Required:

- Infrastructure definition (Terraform or CloudFormation)
- Secure handling of secrets (no plaintext keys)
- Separation of config vs code

Explain:

- Where AI API keys live
- How you would rotate them
- How you'd scale under bursty AI usage

3.2 Containerization (Optional but Strong Signal)

- Dockerize backend (frontend optional)
- Show how you'd deploy it: ECS, EKS, or serverless
- Explain scaling constraints specific to AI workloads

Bonus Sections (Optional, Pick Any)

- Streaming AI responses (token-by-token UX)
- Tool/function calling with the LLM
- Background async processing (queues, workers)
- Cost estimation for 1k / 10k / 100k requests

Deliverables

- Git repository (mono-repo or split)
- README covering:
 - Architecture decisions
 - AI design choices
 - Trade-offs and known limitations
 - Clear instructions to run locally